Operating Systems – COC 3071

SE 5th A – Fall 2025

After-mid Homework -1

Part 1: Semaphore theory

1. **A counting semaphore is initialized to 7. If 10 wait() and 4 signal() operations are performed, find the final value of the semaphore.**

Answer:

- Initial value: S = 7
- Each wait() decreases by 1: 10 wait → 7 - 10 = **-3**
- Each signal() increases by 1: 4 signal → -3 + 4 = **1**

Final value: 1

2. **A semaphore starts with value 3. If 5 wait() and 6 signal() operations occur, calculate the resulting semaphore value.**

Answer:

- Initial: S = 3
- 5 wait → 3 - 5 = -2
- 6 signal → -2 + 6 = 4

Final value: 4

3. **A semaphore is initialized to 0. If 8 signal() followed by 3 wait() operations are executed, find the final value.**

Answer:

- Initial: S = 0
- 8 signal → 0 + 8 = 8
- 3 wait → 8 - 3 = 5

Final value: 5

4. **A semaphore is initialized to 2. If 5 wait() operations are executed:**

a) **How many processes enter the critical section?**

b) **How many processes are blocked?**

Answer:

- Initial: S = 2
- Max 2 processes can enter the critical section (value > 0)
- Remaining 3 wait operations are blocked

a) Processes entered: 2

b) Processes blocked: 3

5. **A semaphore starts at 1. If 3 wait() and 1 signal() operations are performed:**

a) **How many processes remain blocked?**

b) **What is the final semaphore value?**

Answer:

- Initial: S = 1
- 3 wait → 1 - 3 = -2 (so 2 processes blocked)
- 1 signal → -2 + 1 = -1

a) Processes blocked: 1 (still waiting)
b) Final value: -1
Negative semaphore value = number of blocked processes

6.

```
semaphore S = 3;
wait(S); wait(S);
signal(S);
wait(S); wait(S);
```

a) **How many processes enter the critical section?**
b) **What is the final value of S?**
Answer:

Operation	**S value**	**Entry in Critical Section**
Initial (S)	3	
Wait (S)	2	Enters Critical Section
Wait (S)	1	Enters Critical Section
Signal (S)	2	Releases Critical Section
Wait (S)	1	Enters Critical Section
Wait (S)	0	Enters Critical Section

a) Processes entered: 4
b) Final value: 0

7.

```
semaphore S = 1;
wait(S); wait(S);
signal(S);
signal(S);
```

a) **How many processes are blocked?**
b) **What is the final value of S?**
Answer:

Operation	**S value**	**Entry in Critical Section**
Initial (S)	1	
Wait (S)	0	Enters Critical Section
Wait (S)	-1	Blocked one process
Signal (S)	0	Unblock one process
Signal (S)	1	Releases Critical Section

a) Processes blocked: 0 (all processes are unblocked after signal)
b) Final value: 1

8. **A binary semaphore is initialized to 1. Five wait() operations are executed without any signal(). How many processes enter the critical section and how many are blocked?**

Answer:

- Initial: S = 1
- First wait → 0 → enters CS
- Next 4 wait → -1, -2, -3, -4 → blocked

Processes entered CS: 1

Processes blocked: 4

9. **A counting semaphore is initialized to 4. If 6 processes execute wait() simultaneously, how many proceed and how many are blocked?**

Answer:

- Initial: S = 4
- First 4 processes → S = 0 → enter CS
- Last 2 processes → blocked

Processes proceed: 4

Processes blocked: 2

10. **A semaphore S is initialized to 2. wait(S); wait(S); wait(S); signal(S); signal(S); wait(S);**

a) **Track the semaphore value after each operation.**

b) **How many processes were blocked at any time?**

Answer:

Operation	S value	Critical Section
Initial	2	
Wait(S)	1	Enters Critical Section
Wait(S)	0	Enters Critical Section
Wait(S)	-1	Block one process
Signal(S)	0	Unblock one process
Signal(S)	1	Release Critical Section
Wait(S)	0	Enters Critical Section

a) Semaphore after each operation: 2 → 1 → 0 → -1 → 0 → 1 → 0

b) Processes blocked at any time: 1

11. A semaphore is initialized to 0. Three processes execute wait() before any signal(). Later, 5 signal() operations are executed. a) How many processes wake up?

b) What is the final semaphore value?

Answer:

- Initial: S = 0
- 3 wait → -3 → 3 processes blocked
- 5 signal → -3 + 5 = 2

a) Processes wake up: 3

b) Final semaphore value: 2

Part 2: Semaphore Coding

Consider the Producer–Consumer problem using semaphores as implemented in Lab-10 (Lab-plan attached). Rewrite the program in your own coding style, compile and execute it successfully, and explain the working of the code in your own words.

Submission Requirements:

- Your rewritten source code
- A brief description of how the code works
- Screenshots of the program output showing successful execution

Answer:

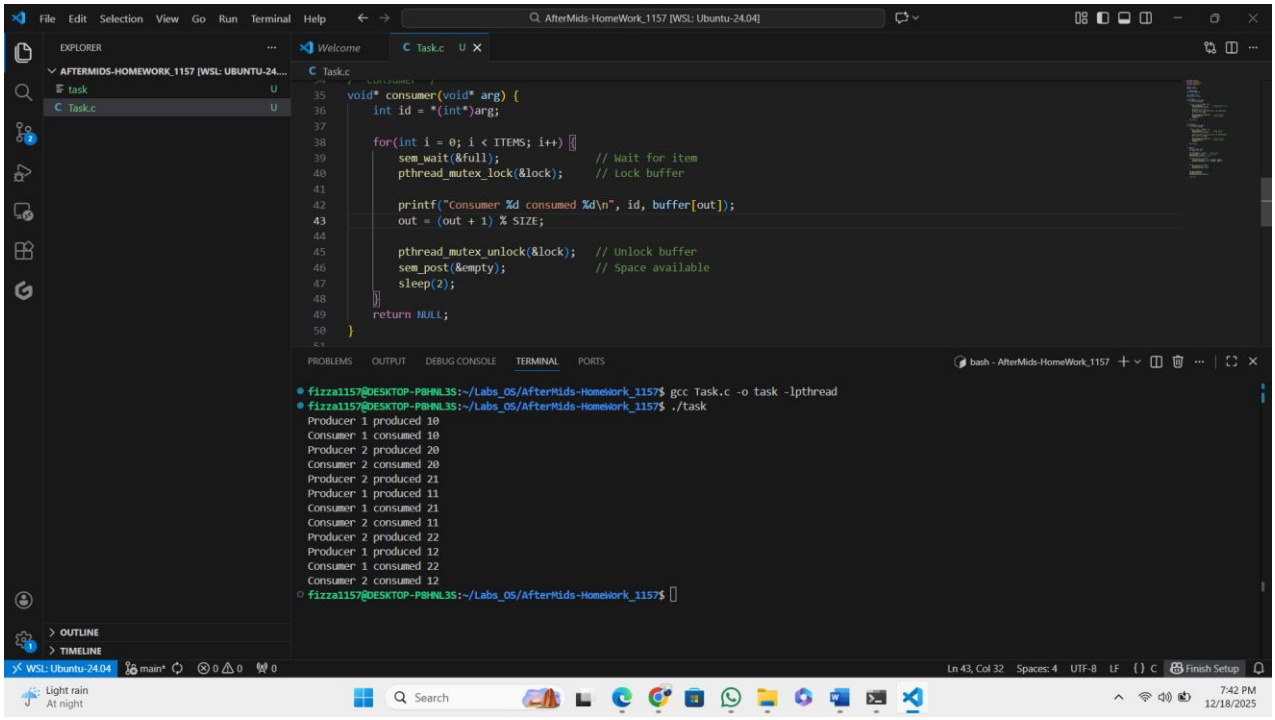
Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define SIZE 5
#define ITEMS 3
int buffer[SIZE];
int in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock;
/* Producer */
void* producer(void* arg) {
   int id = *(int*)arg;
   for(int i = 0; i < ITEMS; i++) {
      sem_wait(&empty);              // Wait for empty space
      pthread_mutex_lock(&lock);     // Lock buffer
      buffer[in] = id * 10 + i;
     printf("Producer %d produced %d\n", id, buffer[in]);
      in = (in + 1) % SIZE;
      pthread_mutex_unlock(&lock);   // Unlock buffer
      sem_post(&full);               // Item available
      sleep(1);
   }
```

```
    return NULL;
}
/* Consumer */
void* consumer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < ITEMS; i++) {
        sem_wait(&full);              // Wait for item
        pthread_mutex_lock(&lock);    // Lock buffer
        printf("Consumer %d consumed %d\n", id, buffer[out]);
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&lock);  // Unlock buffer
        sem_post(&empty);             // Space available
        sleep(2);
    }
    return NULL;
}
int main() {
    pthread_t p[2], c[2];
    int id[2] = {1, 2};
    sem_init(&empty, 0, SIZE);  // Buffer empty
    sem_init(&full, 0, 0);      // No items
    pthread_mutex_init(&lock, NULL);
    for(int i = 0; i < 2; i++) {
        pthread_create(&p[i], NULL, producer, &id[i]);
        pthread_create(&c[i], NULL, consumer, &id[i]);
    }
    for(int i = 0; i < 2; i++) {
        pthread_join(p[i], NULL);
        pthread_join(c[i], NULL);
    }
    sem_destroy(&empty);
    sem_destroy(&full);
```

```
    pthread_mutex_destroy(&lock);
    return 0;
}
```

Output:

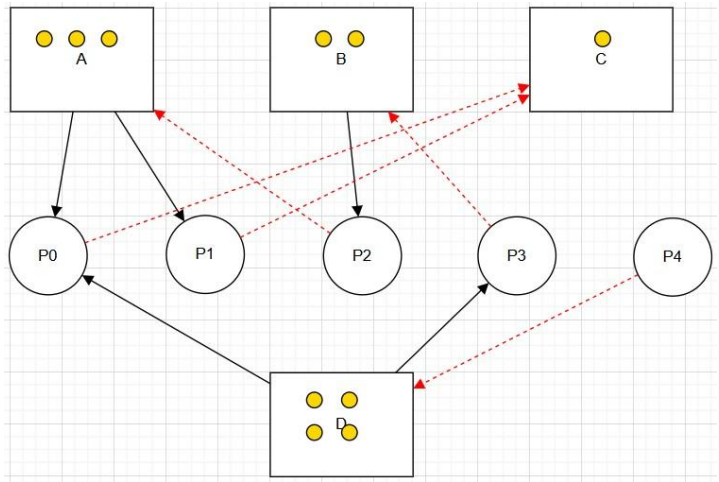


Description Summary:

- Producers put items in the buffer.
- Consumers take items from the buffer.
- Semaphores make sure producers don't overflow the buffer and consumers don't consume from an empty buffer.
- Mutex ensures only one thread changes the buffer at a time.
- The program prints the production and consumption sequence.

Part 3: RAG (Recourse Allocation Graph)

- Convert the following graph into matrix table ,



Solution:

From the diagram:

- A → 3 instances
- B → 2 instances
- C → 1 instance
- D → 5 instances

Identify Allocations

From the graph:

- A → P0 (1 instance of A allocated)
- A → P1 (1 instance of A allocated)
- B → P2 (1 instance of B allocated)
- D → P0 (1 instance of D allocated)
- D → P3 (1 instance of D allocated)

Identify Requests

From the graph:

- P0 → C
- P1 → B,C
- P2 → A
- P3 → B
- P4 → D

Matrix Representation

Allocation Matrix

Process	A	B	C	D
P0	1	0	0	1
P1	1	0	0	0
P2	0	1	0	0
P3	0	0	0	1
P4	0	0	0	0

Request Matrix

Process	A	B	C	D
P0	0	0	1	0
P1	0	0	1	0
P2	1	0	0	0
P3	0	1	0	0
P4	0	0	0	1

Available Resource Vector

Resource	Total	Allocated	Available
A	3	2	1
B	2	1	1
C	1	0	1
D	4	2	2

Part 4: Banker's Algorithm

System Description:

- The system comprises five processes (P0–P3) and four resources (A,B,C,D).

- Total Existing Resources:

Total			
A	B	C	D
6	4	4	2

- Snapshot at the initial time stage:

	Allocation				Max				Need			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	1	1	3	2	1	1				
P1	1	1	0	0	1	2	0	2				
P2	1	0	1	0	3	2	1	0				
P3	0	1	0	1	2	1	0	1				

-

Questions:

1. **Compute the Available Vector**:

    - Calculate the available resources for each type of resource.

Process	**Allocation**				**Max**				**Availability**			
	A	**B**	**C**	**D**	**A**	**B**	**C**	**D**	**A**	**B**	**C**	**D**
P0	2	0	1	1	3	2	1	1	2	2	2	0
P1	1	1	0	0	1	2	0	2	4	2	3	1
P2	1	0	1	0	3	2	1	0	5	2	4	1
P3	0	1	0	1	2	1	0	1	5	3	4	2

2. **Compute the Need Matrix**:

    • Determine the need matrix by subtracting the allocation matrix from the maximum matrix.

Process	**Allocation**				**Max**				**Need**			
	A	**B**	**C**	**D**	**A**	**B**	**C**	**D**	**A**	**B**	**C**	**D**

P0	2	0	1	1	3	2	1	1	1	2	0	0
P1	1	1	0	0	1	2	0	2	0	1	0	2
P2	1	0	1	0	3	2	1	0	2	2	0	0
P3	0	1	0	1	2	1	0	1	2	0	0	0

3. **Safety Check**:

- Determine if the current allocation state is safe. If so, provide a safe sequence of the processes.
- Show how the Available (working array) changes as each process terminates.

Answer:

Initial: Work = [2, 2, 2, 0]

Step 1: Execute P0

Need ≤ Work → YES

Release Allocation (2,0,1,1)

Work = [4, 2, 3, 1]

Step 2: Execute P2

Need ≤ Work → YES

Release Allocation (1,0,1,0)

Work = [5, 2, 4, 1]

Step 3: Execute P3

Need ≤ Work → YES

Release Allocation (0,1,0,1)

Work = [5, 3, 4, 2]

Step 4: Execute P1

Need ≤ Work → YES

Release Allocation (1,1,0,0)

Work = [6, 4, 4, 2]

Safe Sequence

⟨ P0 → P2 → P3 → P1 ⟩

Submission Guidelines:

- Ensure all answers are well-explained and calculations are shown step-by-step.
- Submit your assignment on MS Team and GitHub in a PDF format.
- VIVA based Evaluation so Develop your own solution after getting help.